Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Requirement: Cloud AWS engineer and Networking

1. Design, deploy, and manage scalable, secure, and highly available AWS cloud database solutions (e.g., Amazon RDS, DynamoDB, Aurora).
2. Monitor database performance, troubleshoot issues, and optimize queries to ensure high availability and reliability.
3. Implement backup, recovery, and disaster recovery strategies for cloud databases.
4. Automate database provisioning, configuration, and maintenance tasks using Infrastructure as Code (IaC) tools like AWS CloudFormation or Terraform.
5. Collaborate with development and operations teams to integrate databases with cloud applications and services.
6. Manage database security, including access control, encryption, and compliance with organizational policies.
7. Perform database migrations from on-premises or other cloud platforms to AWS.
8. Develop and maintain documentation for database architecture, configurations, and operational procedures.
9. Stay updated with AWS cloud services and best practices to continuously improve cloud infrastructure and database management.
10. Provide technical support and guidance for cloud infrastructure and database-related incidents and projects.

Detailed explanation of each point, reflecting responsibilities and skills expected from a cloud database engineer or architect, particularly in an AWS (Amazon Web Services) environment:

1. Design, Deploy, and Manage Scalable, Secure, and Highly Available AWS Cloud Database Solutions

This involves:

- **Selecting the right AWS database service** (e.g., Amazon RDS, Aurora, DynamoDB) based on use case (OLTP, analytics, key-value, document store, etc.)
- Designing **high availability (HA)** with features like:
  - **Multi-AZ deployments** (RDS/Aurora)
  - **Read replicas** for load distribution
  - **Global Tables** in DynamoDB for multi-region support
- Implementing **scalability**:
  - Aurora Serverless v2 for auto-scaling
  - DynamoDB auto-scaling based on traffic
  - Partitioning/sharding for large-scale data
- Ensuring **security**:
  - Encryption at rest (KMS integration)
  - Encryption in transit (SSL/TLS)
  - Private access via **VPC peering**, **security groups**, or **AWS PrivateLink**

✅ **Key Tools/Services**:

- RDS, Aurora, DynamoDB, ElastiCache, Neptune
- IAM, KMS, VPC, Route 53

2. Monitor Database Performance, Troubleshoot Issues, and Optimize Queries

Responsibilities include:

- Using tools like:
  - **Amazon CloudWatch** for metrics (CPU, IOPS, connections, storage)
  - **Performance Insights** (Aurora/RDS) for identifying slow queries
  - **Enhanced Monitoring** for deep OS-level stats
- Query tuning:
  - Analyzing **query execution plans**
  - Using **indexes**, **materialized views**, and **caching** where appropriate
- Proactively setting up **alerts** for thresholds (e.g., storage usage > 80%)

✅ **Key Techniques**:

- SQL tuning
- DynamoDB partition key analysis
- RDS/Aurora Performance Insights
- CloudWatch Alarms + SNS

3. Implement Backup, Recovery, and Disaster Recovery (DR) Strategies

You must:

- Configure **automated backups** in RDS/Aurora with defined retention (1–35 days)
- Perform **manual snapshots** for critical events or before upgrades
- Enable **point-in-time recovery (PITR)** to recover data to a specific timestamp
- Set up **cross-region replication** for DR (Aurora Global Databases or snapshot copies)
- Test recovery procedures **regularly**

✅ **DR Planning Includes**:

- RTO (Recovery Time Objective)
- RPO (Recovery Point Objective)
- Failover/failback validation

4. Automate Database Provisioning and Maintenance with IaC (CloudFormation or Terraform)

- Use **Terraform** or **AWS CloudFormation** to:
  - Automate creation of RDS instances, Aurora clusters, parameter groups, subnet groups
  - Manage secrets, IAM roles, and security groups
  - Enforce consistency across dev, staging, and prod environments
- Implement **modules** for reusable patterns
- Integrate with **CI/CD pipelines** for deployments

✅ **IaC Benefits**:

- Version control of infrastructure
- Reduced manual errors
- Easier replication across regions/accounts

5. Collaborate with Dev & Ops Teams to Integrate Databases with Cloud Apps

- Participate in:
  - **Design reviews** for database schemas
  - **Application deployments** that involve data models
- Help developers consume **managed database services** securely and efficiently
- Work with SRE/DevOps to:
  - Automate deployments
  - Implement health checks
  - Configure logging and tracing

✅ **Tools**:

- AWS Lambda, API Gateway, ECS/EKS, Step Functions
- CI/CD: CodePipeline, GitHub Actions, Jenkins

6. Manage Database Security: Access, Encryption, Compliance

- Implement **least privilege access** using IAM roles and policies
- Use **KMS** for encryption at rest

- Enforce **TLS for data in transit**
- Enable **audit logging** (e.g., PostgreSQL's pgaudit, MySQL general logs)
- Comply with standards like **HIPAA, PCI-DSS, ISO 27001**

✅ **Security Services**:

- AWS IAM
- AWS Secrets Manager / Parameter Store
- RDS/Aurora Encryption
- AWS Config + CloudTrail

7. Perform Database Migrations to AWS

- Migrate from:
  - On-prem databases (e.g., Oracle, SQL Server, MySQL)
  - Other cloud providers (Azure, GCP)
- Use **AWS Database Migration Service (DMS)** for:
  - Heterogeneous/homogeneous migrations
  - Minimal downtime migrations (CDC: change data capture)
- Perform **schema conversion** with **AWS SCT (Schema Conversion Tool)** if needed
- Validate data integrity and app compatibility post-migration

✅ **Migration Phases**:

1. Assessment
2. Schema conversion
3. Migration
4. Cutover
5. Post-migration optimization

8. Develop and Maintain Documentation

- Maintain:
  - **Architecture diagrams** for DB systems
  - **Runbooks** for operations and incident response
  - **Backup/restore** procedures
  - **Change history** for configurations
- Document performance benchmarks, schema changes, and DR strategies
- Ensure documentation is accessible to ops/dev teams and updated regularly

✅ Tools:

- Confluence, Markdown repos in GitHub
- AWS Well-Architected Tool for best practices

9. Stay Updated with AWS Services & Best Practices

- Regularly explore:
  - **New database features** (e.g., Aurora Limitless, RDS Blue/Green deployments)
  - **Cost optimization tools** (e.g., Graviton instances, Aurora I/O-Optimized)
  - **Security enhancements**
- Participate in:
  - AWS webinars, re:Invent, AWS blogs
  - AWS certification programs

✅ Use:

- AWS Well-Architected Framework
- Trusted Advisor for recommendations

10. Provide Technical Support & Guidance for DB Incidents & Projects

- Act as escalation point for:
  - Performance issues
  - Failovers
  - Data loss/recovery incidents
- Support project teams with:
  - Designing scalable schemas
  - Ensuring DB meets SLOs
  - Troubleshooting latency or connection problems
- Participate in **postmortems** and **blameless RCA**

✅ Requires:

- Strong communication
- Incident response maturity
- Proactive monitoring and alerting setup

🧠 **Bonus: Certifications to Validate These Skills**

- **AWS Certified Database – Specialty**
- **AWS Certified Solutions Architect – Associate/Pro**
- **Terraform Associate (HashiCorp)**
- **CKA/CKAD** (for Kubernetes + DB ops in containers)

Detailed **resume bullet list**, **training roadmap**, or **interview prep guide**

✅ **1. Resume Bullet List (Cloud Database Engineer / AWS DBA)**

You can use these directly in your resume under a role like *Cloud Database Engineer*, *DevOps DBA*, or *Cloud Infrastructure Specialist*:

- Designed and deployed highly available and scalable AWS database solutions using Amazon RDS, Aurora, and DynamoDB for transactional and analytical workloads.
- Implemented multi-AZ and cross-region replication strategies to ensure high availability and disaster recovery across global environments.
- Tuned SQL and NoSQL workloads for performance using AWS Performance Insights, CloudWatch, and query optimization best practices.
- Automated provisioning and configuration of database infrastructure using Terraform and AWS CloudFormation, reducing manual setup time by 80%.
- Developed robust backup, recovery, and failover processes for critical databases using native AWS tools and point-in-time recovery (PITR).
- Collaborated with DevOps and application teams to integrate secure and high-performing data layers into cloud-native applications on ECS and Lambda.
- Enforced strict access controls, encryption policies, and audit trails using AWS KMS, IAM, and Secrets Manager in compliance with SOC2 and HIPAA standards.
- Led migrations of legacy on-premise databases to AWS using DMS and SCT, achieving zero-downtime transitions for mission-critical systems.

- Maintained detailed operational documentation and architecture diagrams, enabling efficient knowledge transfer and onboarding.
- Provided Tier-3 support for production incidents involving database performance, failovers, and connection issues in cloud-native and hybrid environments.

📘 2. Training Roadmap: AWS Cloud Database Engineer

This roadmap is ideal if you're learning or mentoring someone toward mastering cloud database roles on AWS:

Phase	Focus Area	Key Topics	Tools
1. Fundamentals	AWS core services, networking, IAM	VPC, Subnets, Security Groups, IAM, EC2	AWS Console, CLI
2. Database Services	RDS, Aurora, DynamoDB, ElastiCache	Backup, replication, scaling, read replicas	RDS, Aurora Global, DynamoDB
3. Performance Monitoring	Metrics, insights, alerting	CloudWatch, Enhanced Monitoring, Performance Insights	SNS, CloudTrail
4. Backup & DR	PITR, snapshots, cross-region DR	Backup retention, failover testing, Aurora Global DB	RDS Snapshots, KMS
5. Infrastructure as Code (IaC)	Terraform or CloudFormation	Modules, state management, resource templates	Terraform, CloudFormation
6. Database Security	Access control, encryption, audit logs	KMS, IAM, TLS, Secrets Manager	Security Hub, Config
7. Automation & Integration	CI/CD for DB changes, app integrations	CodePipeline, GitHub Actions, Lambda triggers	ECS, Lambda, Step Functions
8. Migrations	DMS, SCT, validation	Data validation, replication lag, schema conversion	AWS DMS, SCT
9. Documentation & Compliance	Runbooks, architecture, governance	Well-Architected Tool, SOPs, audit trails	Confluence, Markdown
10. Advanced Topics	Multi-region, serverless DBs, cost optimization	Aurora Serverless, DynamoDB Global Tables	AWS Billing, Graviton

📚 Suggested Certifications:

- AWS Certified Database – Specialty
- AWS Certified Solutions Architect – Associate/Professional
- Terraform Associate
- CKA (for Kubernetes integration)

🎯 3. Interview Preparation Guide: AWS Cloud Database Role

Use this to prepare for interviews — categorized by question type:

🔷 Technical Questions

- How would you design a multi-region database deployment in AWS?
- Compare RDS vs Aurora — when would you use each?

- How does DynamoDB scale and what are partition key best practices?
- How do you monitor and tune performance for RDS-based workloads?
- Describe a disaster recovery strategy for a business-critical cloud database.

🔷 **Scenario-Based Questions**

- A production RDS instance is showing high CPU — how do you troubleshoot?
- Your backups are failing silently — walk through how you'd identify the cause.
- A dev team needs a read-only clone of production — how would you provide this securely?
- Describe your steps when migrating a SQL Server from on-prem to AWS.

🔷 **DevOps & Automation**

- How do you use Terraform to manage database resources?
- What's your approach to automating backups and patching?
- How would you integrate DB changes into a CI/CD pipeline?

🔷 **Security & Compliance**

- Explain how you secure access to RDS instances.
- What tools do you use to audit database access and changes?
- How do you ensure compliance with data residency requirements?

🔷 **Behavioral Questions**

- Describe a time you dealt with a production database outage — what did you learn?
- How do you stay updated with AWS changes?
- Tell us about a conflict you had with development/operations — how did you resolve it?